

Presentacion Patrones de Diseño

Principios de Diseño de Software

Lenin Jesus Hernandez Ramirez
Contreras Mo

BUILDER



BUILDER- ANALOGÍA

- Imaginemos que se tiene que construir una casa con su estructura (paredes, puerta, ventanas), pero habrá situaciones donde la casa que tengamos que construir tendrá otros componentes. Crear un objeto tan complejo no puede crearse a partir de nuestra formula de casa original.



Definición

BUILDER es un patrón de diseño que nos permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.

- Tipo de Patron: Creacional
- Alcance: Objeto
- Alias: Sin Alias Conocidos

Intención

“Separar la construcción de un objeto complejo de su representación de modo que el mismo proceso de construcción pueda crear diferentes representaciones.”

Fragmento Extraído de: Gamma et al., 1994

Problema

El problema fundamental que originó la creación del patrón **BUILDER** es la creación de objetos complejos que tienen una inicialización demasiado grande.

Motivación

- Evitar Constructores demasiado Grandes
- Control de la Creación de nuevos Objetos
- Creación de diferentes objetos con el mismo proceso

(Gamma et al., 1994)

Principios de Diseño

- Principio de Responsabilidad Única: Separa la creación del objeto de la lógica de negocio del objeto.
- Programar para interfaces: El Director no se encuentra acoplado a un constructor en específico (Bajo Acoplamiento)
- Encapsular lo que varía: La interfaz permite al constructor ocultar la representación y la estructura interna del producto.

Participantes y Responsabilidades

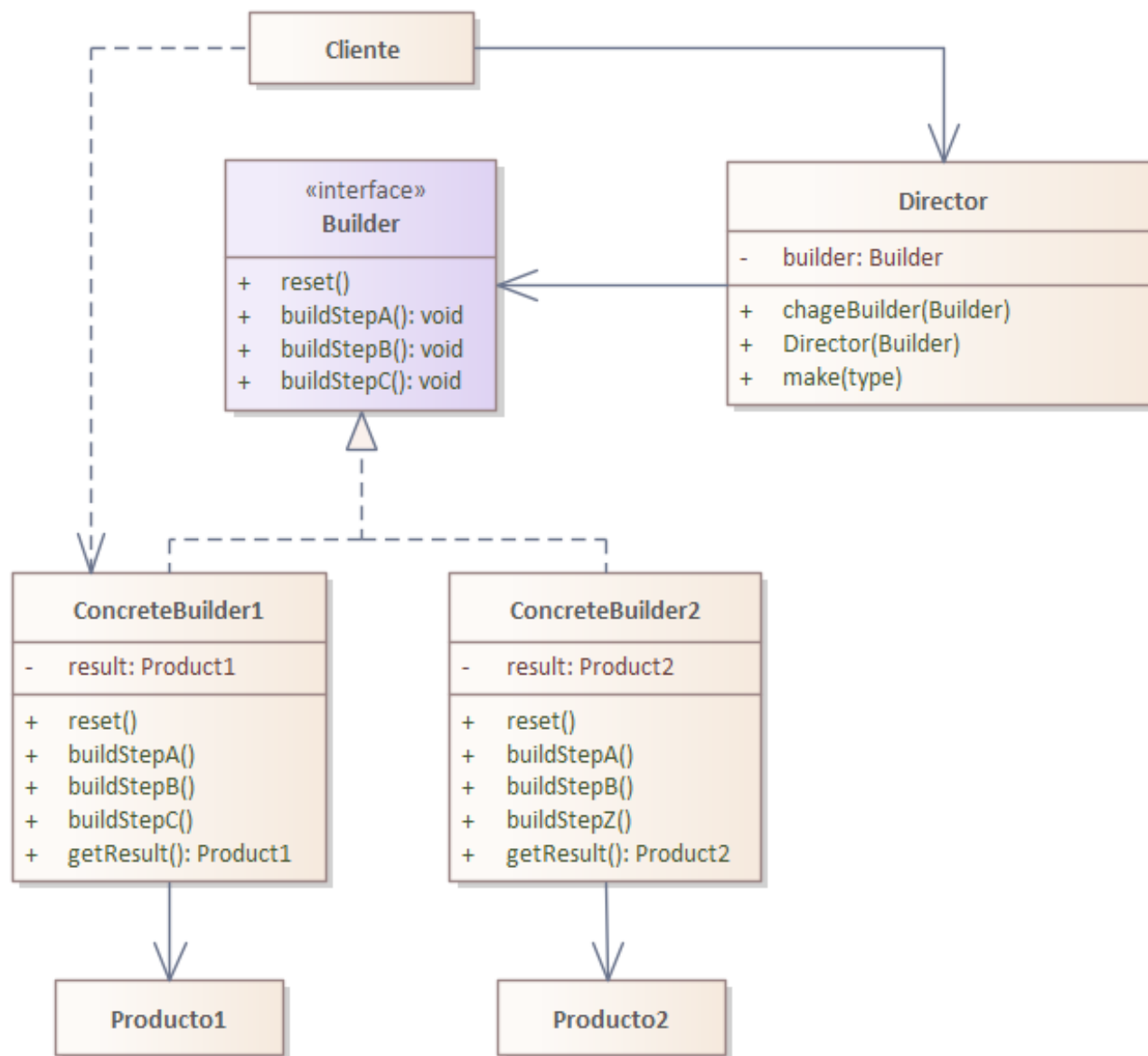
- Cliente: Crea al director y a sus métodos.
- Director: Construye un objeto utilizando la interfaz Builder.
- Interfaz Builder: Especifica una interfaz abstracta para crear las partes de un objeto Product
- Builder Concreto: Construye y ensambla las partes del producto mediante la implementación de la interfaz Builder. Define y mantiene el rastro de la representación que crea.
- Producto: Representa el objeto complejo bajo construcción. El ConcreteBuilder construye la representación interna del producto y define el proceso mediante el cual se ensambla.

(Gamma et al., 1994)

Diagrama de Clases

Extraído y Adaptado de
Refactoring.Guru, 2026

class Builder-Diagrama de Clases



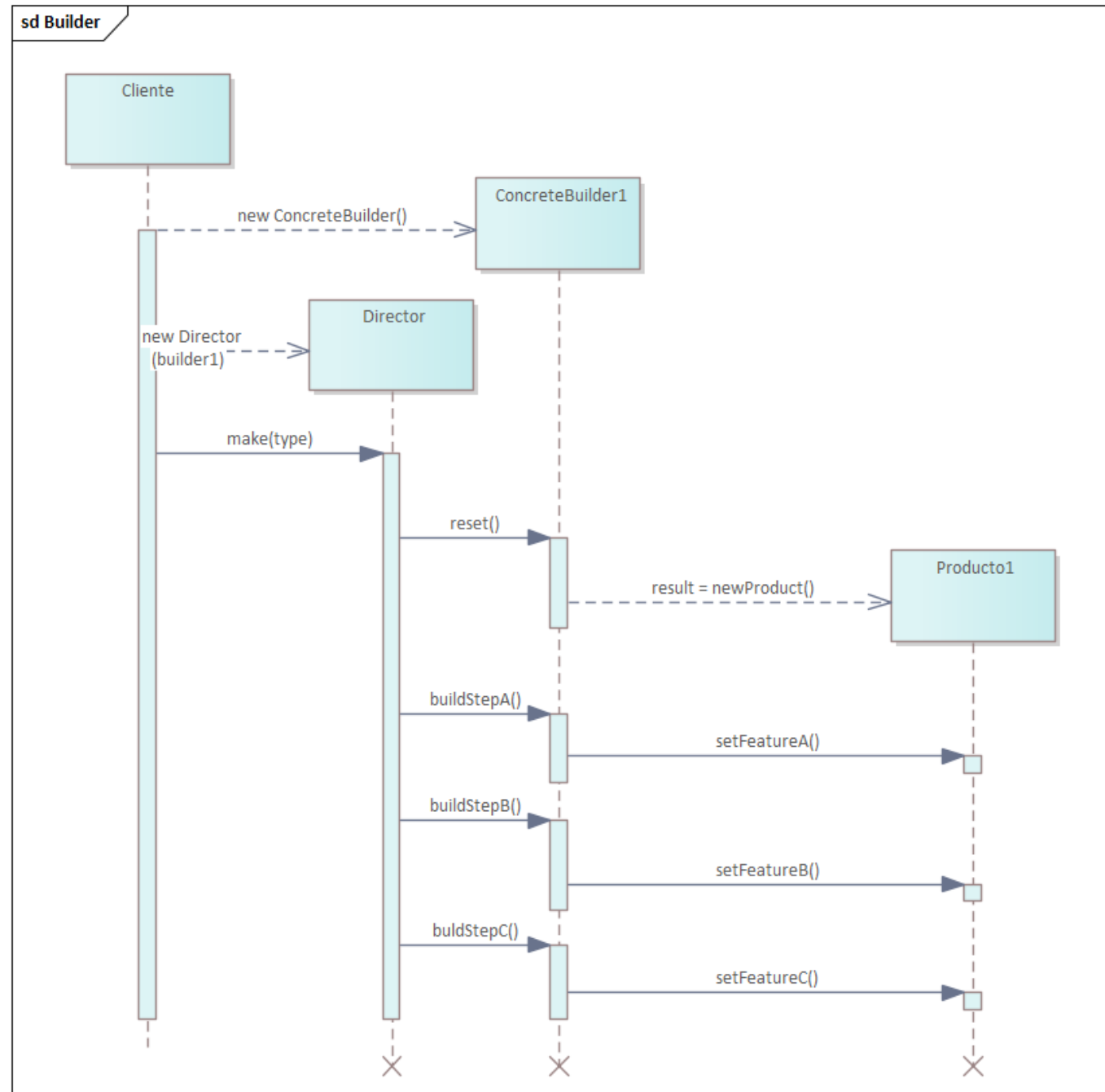
Relaciones entre los Participantes

- El cliente crea el objeto Director y lo configura con el objeto Builder deseado.
- El Director notifica al constructor (builder) cada vez que se debe construir una parte del producto.
- El Builder maneja las solicitudes del director y añade las partes al producto.
- El cliente recupera el producto desde el constructor (builder).

(Gamma et al., 1994)

Diagrama de Secuencia

Diagrama de secuencia adaptado de Shvets (2026) y basado en la estructura conceptual de los participantes de Gamma et al. (1994)



VENTAJAS	DESVENTAJAS
• Se puede reutilizar el código para construir varias representaciones de productos, favoreciendo composición sobre herencia (ya que no creamos más subclases) • Puedes gestionar objetos durante su creación • Aisla código complejo de la lógica de negocio, respetando el Principio de Responsabilidad Única.	• Aumento en la Complejidad del código • Dificultad de lectura del código

(Refactoring.Guru, 2026)

Usos Conocidos

- Implementación de DTO (Data Transfer Object)
- Configuración de Perfil para creación gradual y personalizada del usuario
- Generación de Reportes y Documentos
- Creación de Pruebas unitarias

PATRONES RELACIONADOS

- **Composite:** Se utiliza para crear arboles complejos para el patrón composite porque los pasos para su construcción se utilizan de forma recursiva.

(Gamma et al., 1994)

CONDICIONES DE USO

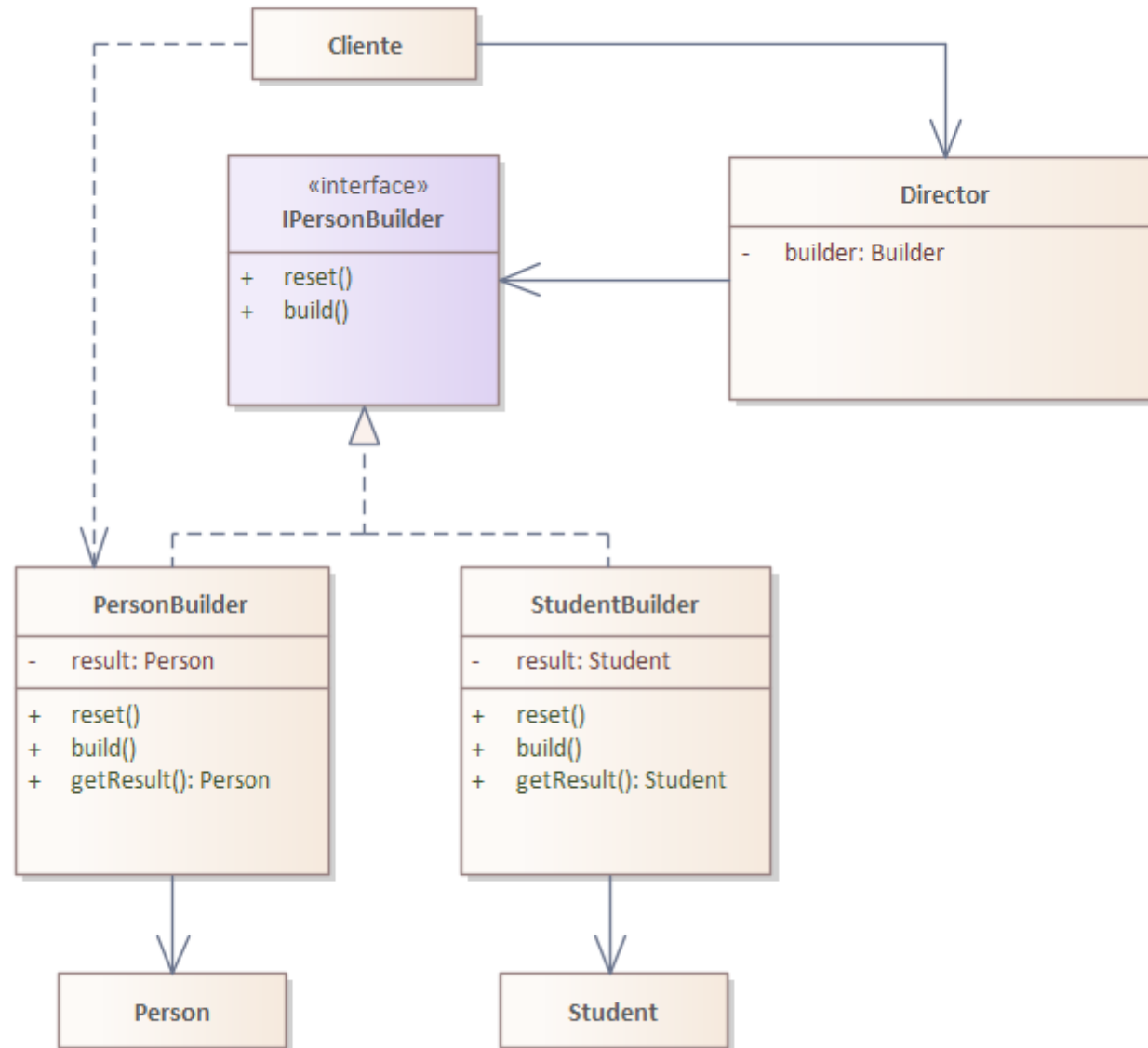
¿CUÁNDO USARLO?	¿CUÁNDO NO?
• El algoritmo para crear un objeto complejo debe ser independiente de la partes que componen el objeto y cómo se ensamblan. • El proceso de construcción debe permitir diferentes representaciones del objeto eso está construido.	• Cuando el constructor sea pequeño • Cuando sólo se definan el constructor vacío y el parametrizado • Si queremos separar atributos sólo para implementar el <i>Builder</i> caeríamos en el anti-patrón

(Gamma et al., 1994)

IMPLEMENTACION DEL PATRON BUILDER

- **Problemática:** En un Sistema para la Universidad Veracruzana se requiere registrar distintos tipos de personas, como personas comunes y estudiantes. Cada tipo posee atributos diferentes y algunos campos son opcionales.

class Implementacion-Diagrama de Clases



CODIGO IMPLEMENTADO

```
1 public interface IPersonBuilder<T>
2 {
3     T Build();
4 }
```

Interfaz: IPersonBuilder

```
using System;

0 references
public class Director
{
    0 references
    public Person CrearPersona()
    {
        return new PersonaBuilder()
            .SetNombre("Jose Maria")
            .SetApellidoPaterno("Contreras")
            .SetApellidoMaterno("Mota")
            .SetEmail("correochema@gmail.com")
            .SetTelefono("2281874642")
            .SetFechaCreacion(DateTime.Now)
            .Build();
    }

    0 references
    public Student CrearEstudiante()
    {
        return new StudentBuilder()
            .SetNombre("Lenin Jesus")
            .SetApellidoPaterno("Hernandez")
            .SetApellidoMaterno("Ramirez")
            .SetMatricula("S24012345")
            .SetEmail("zS24012345@estudiantes.uv.mx")
            .SetFechaCreacion(DateTime.Now)
            .Build();
    }
}
```

Clase Director

9 references

```
public class PersonBuilder : IPersonBuilder<Person>
```

```
{
```

3 references

```
public string Nombre { get; private set; } = "";
```

3 references

```
public string ApellidoPaterno { get; private set; } = "";
```

3 references

```
public string ApellidoMaterno { get; private set; } = "";
```

3 references

```
public string Email { get; private set; } = "";
```

3 references

```
public string Telefono { get; private set; } = "";
```

5 references

```
public DateTime FechaCreacion { get; private set; }
```

1 reference

```
public PersonBuilder()
```

```
{
```

```
    Reset();
```

```
}
```

3 references

```
public void Reset()
```

```
{
```

```
    Nombre = "";
```

```
    ApellidoPaterno = "";
```

```
    ApellidoMaterno = "";
```

```
    Email = "";
```

```
    Telefono = "";
```

```
    FechaCreacion = default;
```

```
}
```

1 reference

```
public PersonBuilder SetNombre(string nombre)
```

```
{
```

```
    Nombre = nombre;
```

```
    return this;
```

```
}
```

1 reference

```
public PersonBuilder SetApellidoPaterno(string apellidoPaterno)
```

```
{
```

```
    ApellidoPaterno = apellidoPaterno;
```

```
    return this;
```

```
}
```

1 reference

```
public PersonBuilder SetApellidoMaterno(string apellidoMaterno)
```

```
{
```

```
    ApellidoMaterno = apellidoMaterno;
```

```
    return this;
```

```
}
```

1 reference

```
public PersonBuilder SetEmail(string email)
```

```
{
```

```
    Email = email;
```

```
    return this;
```

```
}
```

1 reference

```
public PersonBuilder SetTelefono(string telefono)
```

```
{
```

```
    Telefono = telefono;
```

```
    return this;
```

```
}
```

1 reference

```
public PersonBuilder SetFechaCreacion(DateTime fechaCreacion)
```

```
{
```

```
    FechaCreacion = fechaCreacion;
```

```
    return this;
```

```
}
```

2 references

```
public Person Build()
```

```
{
```

```
    if (FechaCreacion == default)
```

```
    {
```

```
        FechaCreacion = DateTime.Now;
```

```
    }
```

```
    Person person = new Person(this);
```

```
    Reset();
```

```
    return person;
```

```
}
```

1 reference

```
public class Person
{
    3 references
    private readonly string nombre;
    3 references
    private readonly string apellidoPaterno;
    3 references
    private readonly string apellidoMaterno;
    2 references
    private readonly string email;
    2 references
    private readonly string telefono;
    2 references
    private readonly DateTime fechaCreacion;

    0 references
    public Person(PersonBuilder builder)
    {
        nombre = builder.Nombre;
        apellidoPaterno = builder.ApellidoPaterno;
        apellidoMaterno = builder.ApellidoMaterno;
        email = builder.Email;
        telefono = builder.Telefono;
        fechaCreacion = builder.FechaCreacion;
    }

    0 references
    public string GetNombreCompleto()
    {
        if (string.IsNullOrEmpty(apellidoMaterno))
        {
            return $"{nombre} {apellidoPaterno}";
        }

        return $"{nombre} {apellidoPaterno} {apellidoMaterno}";
    }
}
```

0 references

```
public string GetEmail()
{
    return email;
}
```

0 references

```
public string GetTelefono()
{
    return telefono;
}
```

0 references

```
public DateTime GetFechaCreacion()
{
    return fechaCreacion;
}
```

9 references

```
public class StudentBuilder : IPersonBuilder<Student>
{
    3 references
    public string Nombre { get; private set; } = "";
    3 references
    public string ApellidoPaterno { get; private set; } = "";
    3 references
    public string ApellidoMaterno { get; private set; } = "";
    3 references
    public string Matricula { get; private set; } = "";
    3 references
    public string Email { get; private set; } = "";
    5 references
    public DateTime FechaCreacion { get; private set; }

    1 reference
    public StudentBuilder()
    {
        Reset();
    }

    3 references
    public void Reset()
    {
        Nombre = "";
        ApellidoPaterno = "";
        ApellidoMaterno = "";
        Matricula = "";
        Email = "";
        FechaCreacion = default;
    }

    1 reference
    public StudentBuilder SetNombre(string nombre)
    {
        Nombre = nombre;
        return this;
    }
}
```

1 reference

```
public StudentBuilder SetApellidoPaterno(string apellidoPaterno)
{
    ApellidoPaterno = apellidoPaterno;
    return this;
}
```

1 reference

```
public StudentBuilder SetApellidoMaterno(string apellidoMaterno)
{
    ApellidoMaterno = apellidoMaterno;
    return this;
}
```

1 reference

```
public StudentBuilder SetMatricula(string matricula)
{
    Matricula = matricula;
    return this;
}
```

1 reference

```
public StudentBuilder SetEmail(string email)
{
    Email = email;
    return this;
}
```

1 reference

```
public StudentBuilder SetFechaCreacion(DateTime fechaCreacion)
{
    FechaCreacion = fechaCreacion;
    return this;
}
```

2 references

```
public Student Build()
{
    if (FechaCreacion == default)
    {
        FechaCreacion = DateTime.Now;
    }

    Student student = new Student(this);

    Reset();

    return student;
}
```

```

0 references
public class Student
{
    3 references
    private readonly string nombre;
    3 references
    private readonly string apellidoPaterno;
    3 references
    private readonly string apellidoMaterno;
    2 references
    private readonly string matricula;
    2 references
    private readonly string email;
    2 references
    private readonly DateTime fechaCreacion;

    0 references
    public Student(StudentBuilder builder)
    {
        nombre = builder.Nombre;
        apellidoPaterno = builder.ApellidoPaterno;
        apellidoMaterno = builder.ApellidoMaterno;
        matricula = builder.Matricula;
        email = builder.Email;
        fechaCreacion = builder.FechaCreacion;
    }

    0 references
    public string GetNombreCompleto()
    {
        if (string.IsNullOrEmpty(apellidoMaterno))
        {
            return $"{nombre} {apellidoPaterno}";
        }
        return $"{nombre} {apellidoPaterno} {apellidoMaterno}";
    }
}

```

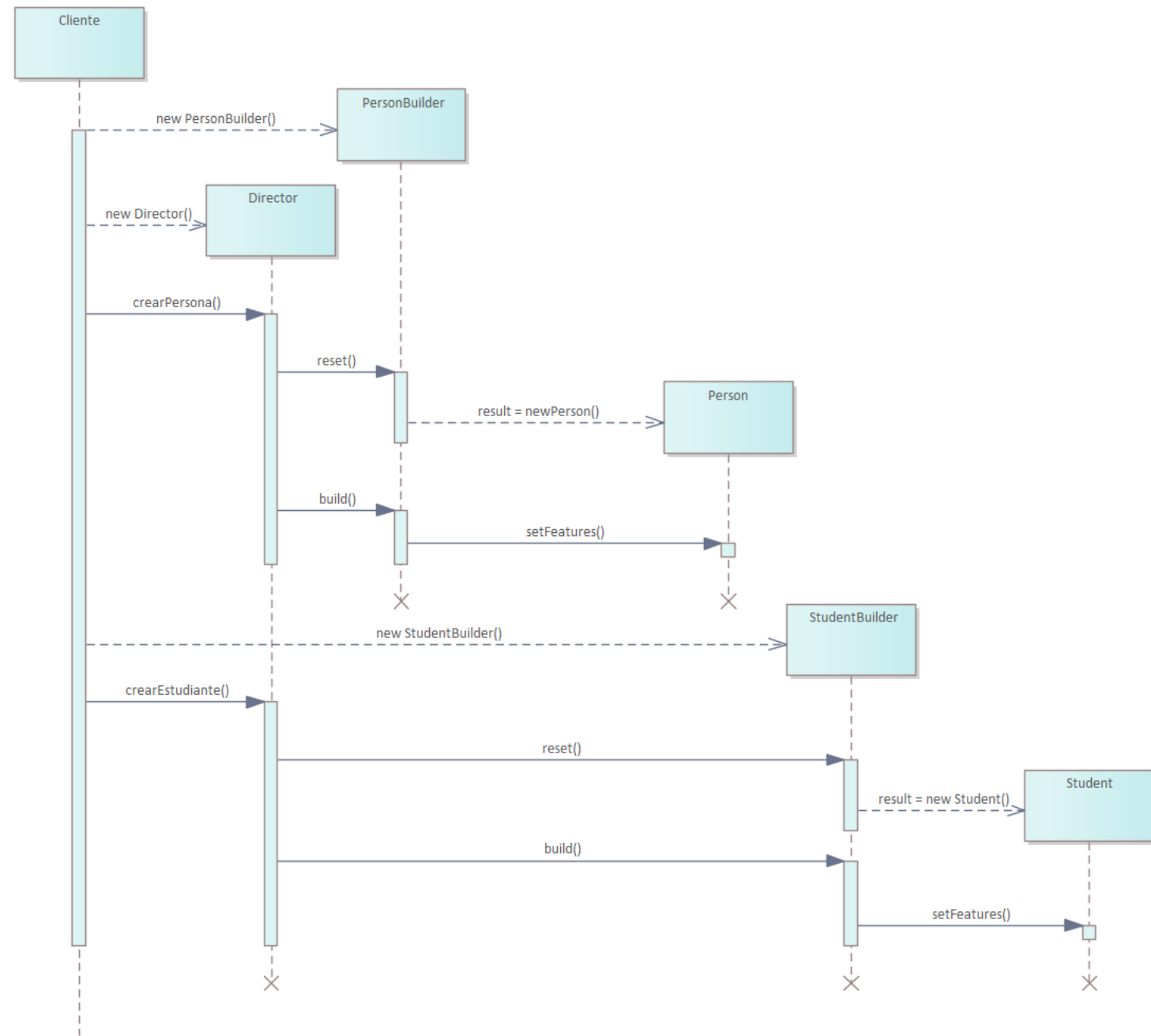
```

0 references
public string GetMatricula()
{
    return matricula;
}

0 references
public string GetEmail()
{
    return email;
}

0 references
public DateTime GetFechaCreacion()
{
    return fechaCreacion;
}

```

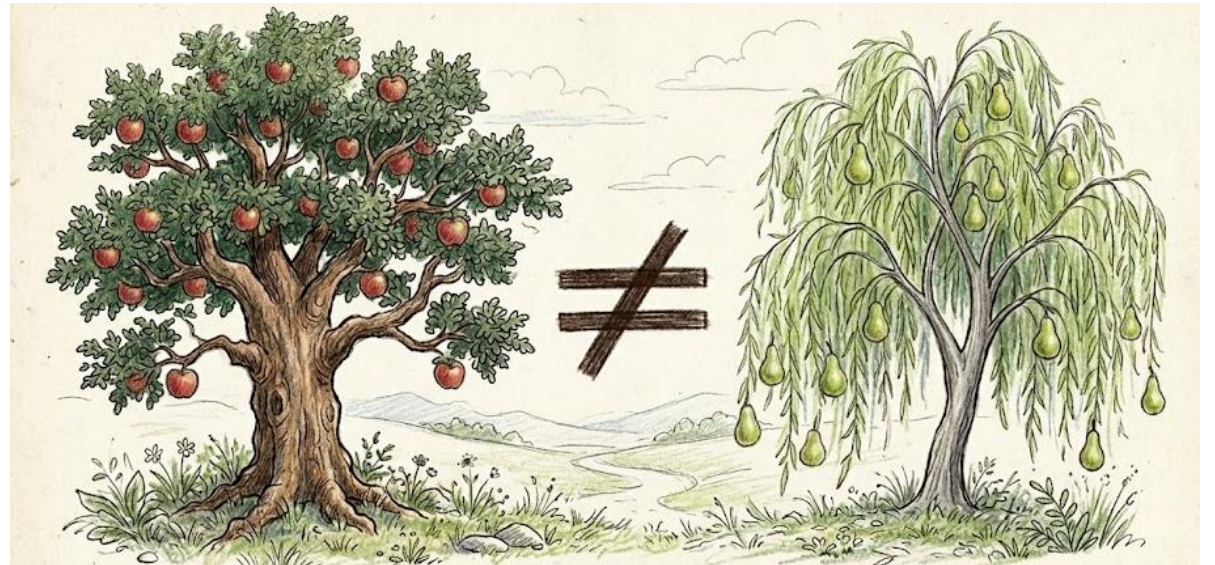


ABSTRACT FACTORY



ABSTRACT FACTORY – ANALOGÍA

- Imaginemos que queremos un árbol de fruta en nuestro patio pero existen tantas variedades de árbol y cada tipo de árbol tiene ciertos cuidados diferentes. Por lo tanto no puedes plantar cualquier árbol en tu casa debido a los cuidados específicos de cada tipo.



Definicion

ABSTRACT FACTORY es un patrón de diseño que nos permite producir familias de objetos relacionados sin especificar sus clases concretas.

- Tipo de Patron: Creacional
- Alcance: Objeto
- Alias: KIT

Intención

“Proporcionar una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.”

Fragmento Extraído de: Gamma et al., 1994

Problema

- La problemática fundamental que ayudó a crear el patrón **ABSTRACT FACTORY** fue la necesidad de manejar productos de categorías similares pero de diferentes clases.

(Refactoring.Guru, 2026)

Motivación

- Evitar Constructores demasiado Grandes
- Control de la Creación de nuevos Objetos
- Creación de diferentes objetos con el mismo proceso

(Gamma et al., 1994)

Principios de Diseño

- Programar para interfaces, no para implementaciones: No creamos productos dependiendo de la decisión del cliente, si no que delegamos esa tarea a las fábricas, lo cual permite crear más productos de los establecidos inicialmente.
- Encapsulamiento: Oculta las fabricas del cliente sin importar su instancia (lo que varía).

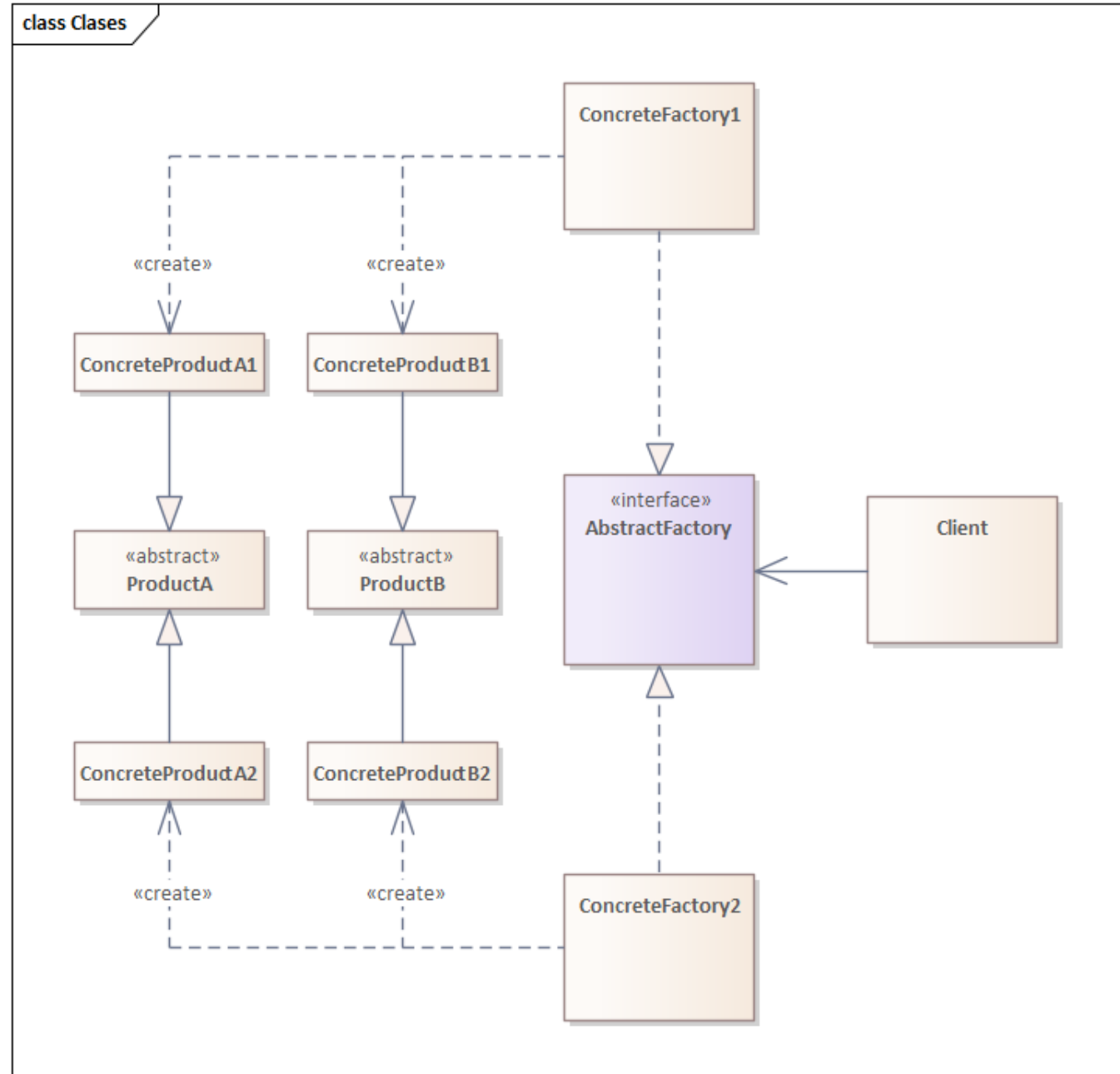
Participantes y Responsabilidades

- Cliente: interactúa con la fábrica indicando los productos que desea crear.
- AbstractFactory: declara operaciones para crear productos.
- ConcreteFactory: implementa los métodos de creación de productos concretos
- AbstractProduct: define las características base que toma un producto concreto
- ConcreteProduct: es el producto que hereda las características comunes, y es creado por la ConcreteFactory

(Gamma et al., 1994)

Diagrama de Clases

Diagrama de Clases de Abstract Factory, por Shvets, A., s.f., Refactoring.Guru



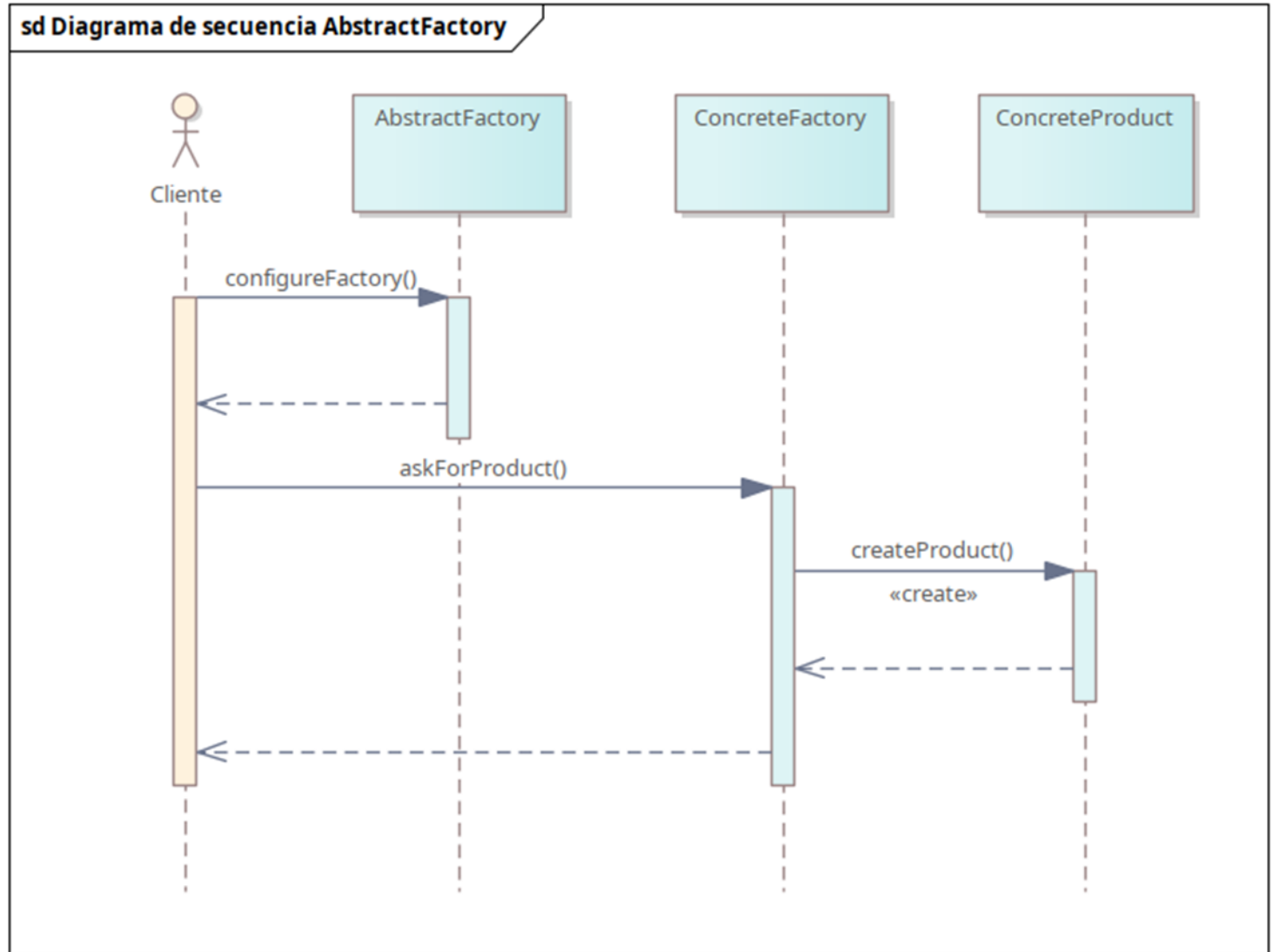
Relaciones entre los Participantes

- Normalmente, se crea una única instancia de una clase ConcreteFactory en tiempo de ejecución. Esta fábrica concreta crea objetos de producto que tienen una implementación particular. Para crear objetos de producto diferentes, los clientes deben utilizar una fábrica concreta diferente.
- AbstractFactory delega la creación de objetos de producto a su subclase ConcreteFactory.

(Gamma et al., 1994)

Diagrama de Secuencia

Diagrama de secuencia adaptado del Patrón de Diseño Abstract Factory, por ReactiveProgramming.io, 2021



Ventajas	Desventajas
• Engloba objetos con características similares para evitar la duplicación de código. • El cliente sólo interactúa con la fábrica. • Mantiene la cohesión entre los objetos resultantes. • Parte del diseño queda desacoplada: La lógica de construcción que ve el cliente de la instanciación real.	• La abstracción puede complicarse cuando no se encuentran características en común • Sólo aplica a situaciones donde se agrupan objetos.

(Refactoring.Guru, 2026)

Usos Conocidos

- Elegir llamar a la implementación correcta de FileSystemAcmeService o DatabaseAcmeService o NetworkAcmeService en tiempo de ejecución.
- Escribir test unitarios se hace mucho más sencillo.
- Cuando te conectas a una aplicación web, esta detecta tu sistema operativo, eligiendo así la apariencia de los componentes. (Ejemplo de construcción de un conversor de formato RTF. Adaptado de Gamma et al. (1994, p. 104).")

(Gamma et al., 1994)

PATRONES RELACIONADOS

- **Factory method:** Es patrón precede para la creación del patrón Abstract Factory donde también se implementan métodos.
- **Singleton:** En el caso de AbstractFactory porque sólo necesitamos una interfaz con la que el cliente interactúe.
- **Prototype:** Se puede utilizar para definir los métodos de las clases de Abstract Factory

(Gamma et al., 1994)

CONDICIONES DE USO

¿CUÁNDO USARLO?	¿CUÁNDO NO?
• Un sistema deba ser independiente de cómo se crean, componen y representan sus productos. • Un sistema deba ser configurado con una de entre múltiples familias de productos. • Una familia de objetos de producto relacionados esté diseñada para ser utilizada en conjunto, y usted necesite hacer cumplir esta restricción. • Se requiera proporcionar una biblioteca de clases de productos y quiera revelar solo sus interfaces, no sus implementaciones.	• No se recomienda utilizarse cuando sólo hay un tipo de objetos en juego, si llegamos a categorizar por el mero hecho de aplicar un patrón, se torna en un antipatrón.

(Gamma et al., 1994)

IMPLEMENTACION DEL PATRON ABSTRACT FACTORY

- **Problemática:** Tienes 42 años, te ganaste la lotería, deseas unos muebles nuevos, pero quieres cohesión en su estilo, victoriano o minimalista, he ahí el dilema.
No te interesa el producto abstracto que es la base de quién sabe qué, sólo quieres que tu silla y tu mesa tengan el estilo de tu elección.

Diagrama de clases de la implementación

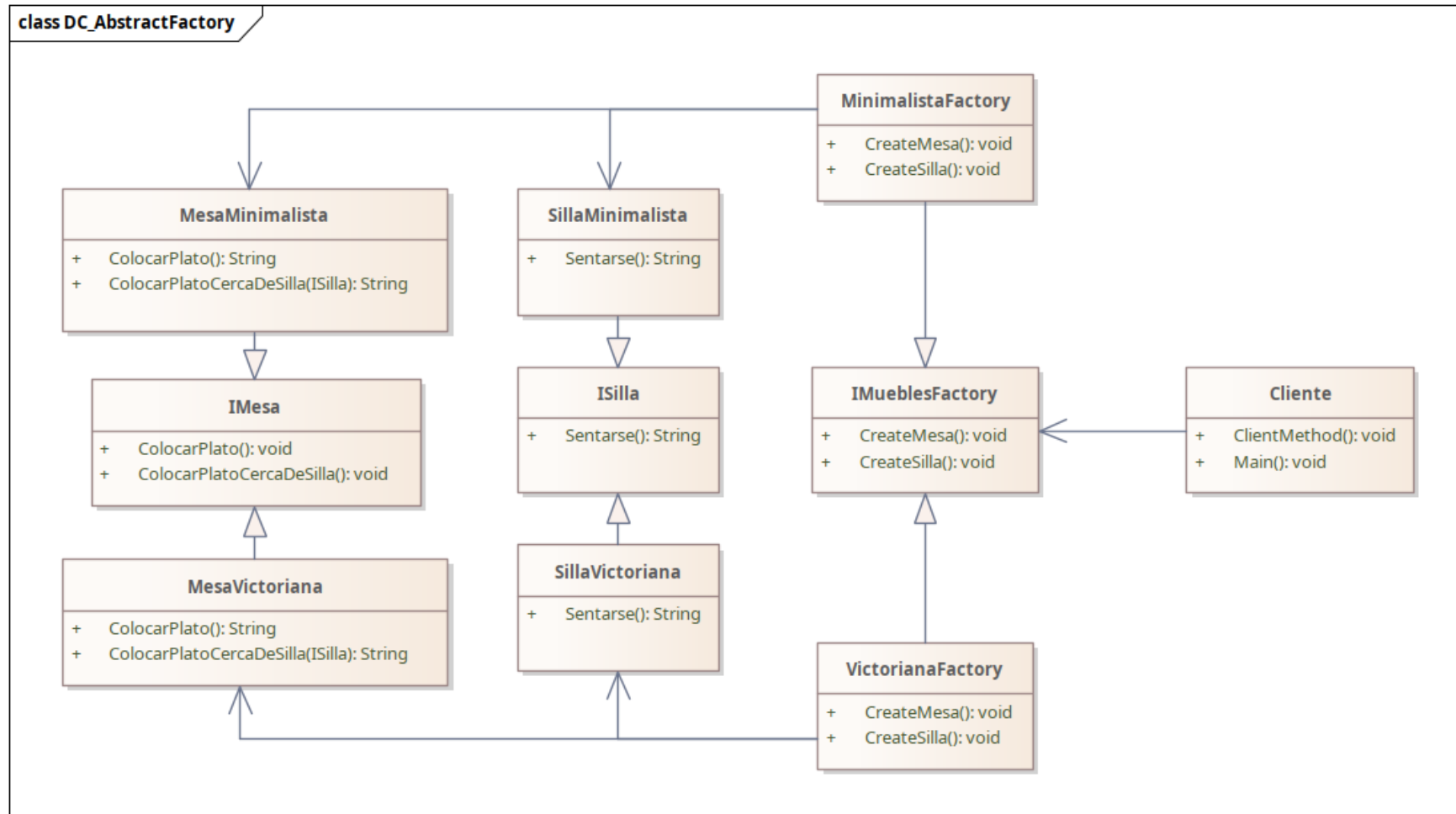


Diagrama de Clases de Abstract Factory en nuestra problemática, realizado por Lenin.

ABSTRACT FACTORY – IMPLEMENTACIÓN

```
class Client
{
    1 reference
    public void Main()
    {
        Console.WriteLine("--- Tienda de Muebles ---");
        Console.WriteLine("1. Minimalista");
        Console.WriteLine("2. Victoriano");
        Console.Write("Seleccione el estilo de su habitación: ");

        string opcion = Console.ReadLine();
        IMueblesFactory factory = null;

        // El switch decide qué fábrica concreta utilizar según la elección
        switch (opcion)
        {
            case "1":
                factory = new MinimalistaFactory();
                break;
            case "2":
                factory = new VictorianaFactory();
                break;
            default:
                Console.WriteLine("Opción no válida, usando Minimalista por defecto.");
                factory = new MinimalistaFactory();
                break;
        }

        Console.WriteLine("\nClient: Configurando la habitación con la fábrica seleccionada...");
        ClientMethod(factory);
    }
}
```

```
1 reference
public void ClientMethod(IMueblesFactory factory)
{
    var silla = factory.CreateSilla();
    var mesa = factory.CreateMesa();

    Console.WriteLine(mesa.ColocarPlato());
    Console.WriteLine(mesa.ColocarPlatoCercaDeSilla(silla));
}
```

```
// AbstractFactory declara métodos para la creación de productos (muebles)
4 references
public interface IMueblesFactory
{
    3 references
    ISilla CreateSilla();

    3 references
    IMesa CreateMesa();
}

// Concrete Factories implementan los métodos de creación
// pero devolviendo productos concretos (estilo Minimalista)
2 references
class MinimalistaFactory : IMueblesFactory
{
    2 references
    public ISilla CreateSilla()
    {
        return new SillaMinimalista();
    }

    2 references
    public IMesa CreateMesa()
    {
        return new MesaMinimalista();
    }
}
```

Resultado esperado al elegir el estilo minimalista:
 Client: Configurando la habitación con la fábrica seleccionada...

Has colocado un plato sobre una mesa de cristal minimalista.

Mesa Minimalista: Colocando plato mientras (Te has sentado en una silla minimalista de líneas rectas.)

```
// Los productos base se representan como interfaces
8 references
public interface ISilla
{
    4 references
    string Sentarse();
}

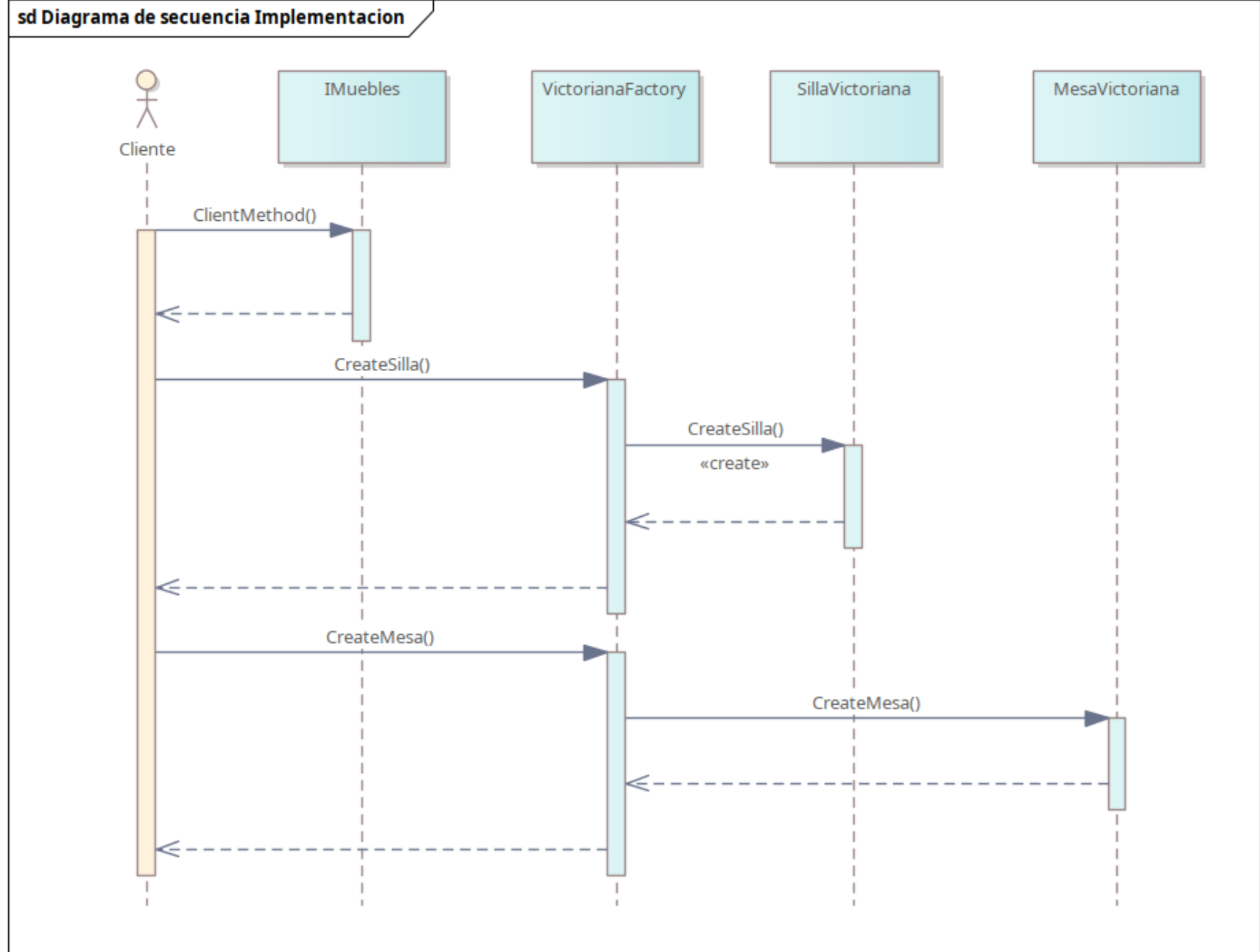
// Los Concrete Products implementan la base
1 reference
class SillaMinimalista : ISilla
{
    3 references
    public string Sentarse()
    {
        return "Te has sentado en una silla minimalista de líneas rectas.";
    }
}

// Los ConcreteProduct implementan los métodos abstractos de sus bases
1 reference
class MesaMinimalista : IMesa
{
    2 references
    public string ColocarPlato()
    {
        return "Has colocado un plato sobre una mesa de cristal minimalista.";
    }

    // Aquí se ilustra la compatibilidad brindada por la relación de los objetos base
    2 references
    public string ColocarPlatoCercaDeSilla(ISilla collaborator)
    {
        var result = collaborator.Sentarse();

        return $"Mesa Minimalista: Colocando plato mientras ({result})";
    }
}
```

Diagrama de secuencia de la implementación



COMPARACION ENTRE LOS PATRONES

BUILDER	ABSTRACT FACTORY
• Dedicado a la creación de objetos complejos • Crea sus objetos a partir de una clase director que maneja el constructor	• Dedicado a la creación de objetos con especificaciones complejas

- Ambos Siguen el Principio de “Encapsular lo que varia” y “Programar Para Interfaces”
- Ambos aumentan la complejidad del código
- Ambos Son Patrones **Creacionales** de **Objeto**

Implementación de IA en la Presentación



Referencias

- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Shvets, A. (2026). *Builder*. Refactoring.Guru.
<https://refactoring.guru/es/design-patterns/builder>